Convergence of MCMC chains

José G. Dias

Table of contents

1. Summary

This text illustrates the work horses of Bayesian estimation, the MCMC algorithms using the estimation of a bivariate normal distribution parameters. We start from the Gibbs sampler that provides an intuitive understanding of the process underlying sampling estimation (see previous text). Then, we focus on the convergence diagnostics of the process.

2. Bayesian estimation of a bivariate model

2.1 Synthetic data set

This is the first example using data (y) in estimation as before we were using MCMC to simulate data from a distribution. We start with synthetic data to show and understand the recovery properties of the parameters (true values). The true model is

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \sim N\left[\begin{pmatrix} -1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 & 0.5 \\ 0.5 & 3 \end{pmatrix}\right]$$

from which we generate a sample of size $n = 1000$.

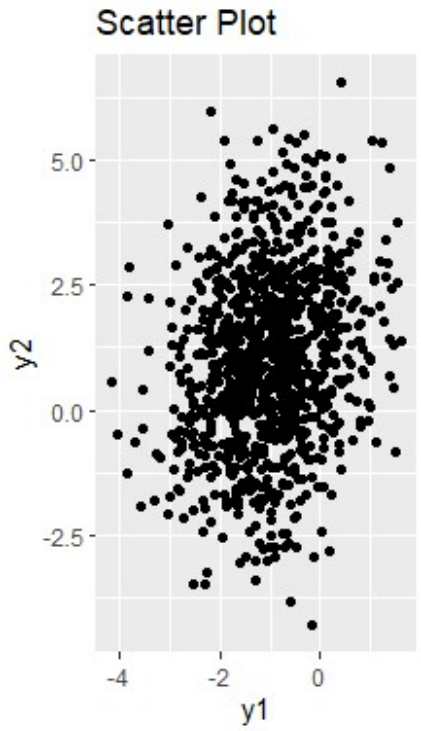
```
library(ggplot2)
library(MASS) # Make sure the MASS package is installed

bivnormalsim <- function(n, mu, sigma2){

  # Generate random samples
  data <- mvrnorm(n, mu = mu, Sigma = sigma2)

return(data)}

# Initialization
set.seed(123)
n      <- 1000
mu     <- c(-1, 1)
sigma2 <- matrix(c(1, 0.5, 0.5, 3), nrow = 2)
```

```r
# Generate random samples from a bivariate normal distribution
y <- bivnormalsim(n, mu, sigma2)

# Print the generated sample
print(head(y))
```

```
           [,1]       [,2]
[1,] -0.3172153 -0.1780718
[2,] -0.1428553  0.3800375
[3,] -0.3512031  3.6748523
[4,] -0.8505859  1.0926537
[5,]  1.3825707  0.6721221
[6,] -1.2552960  4.1719624
```

and the scatter plot is:

```r
xy <- as.data.frame(y)
ggplot(xy, aes(x = V1, y = V2)) +
  geom_point() +
  labs(title = "Scatter Plot", x = "y1", y = "y2")+
  coord_fixed(ratio = 1, xlim = NULL, ylim = NULL, expand = TRUE, clip =
"on")
```



3. Model specification

The model specification is given by

$$y|\mu, \Sigma \sim N_2(\mu, \Sigma)$$

with priors distributions

$$\mu \sim N_2(\mu_0, \Delta_0)$$

$\Sigma \sim IW(\nu_0, S_0^{-1})$,

where IW is the inverse-Wishart, a generalization of the inverse-Gamma distribution.

In this example, let us assume the following values for the prior parameters:

$$\mu_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\Delta_0 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$$

$$\nu_0 = 1$$

$$S_0^{-1} = I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

```
# Prior parameters

prior_mean_mean <- c(0, 0)
prior_mean_cov  <- diag(2,nrow=2)
prior_cov_df    <- 1
prior_cov_cov   <- diag(1,nrow=2)
```

Then, the conditional distribution for the mean is

$\mu|\Sigma, y \sim N(\mu_n, \Delta_n)$,

with

$$\mu_n = (n\Sigma^{-1} + \Delta_0^{-1})^{-1} \left(\Sigma^{-1} \sum_{i=1}^{n} y_i + \Delta_0^{-1} \mu_0 \right)$$

and

$\Delta_n = (n\Sigma^{-1} + \Delta_0^{-1})^{-1}$.

And for the Σ, the conditional distribution is

$$\Sigma|\mu, y \sim IW\left(\nu_0 + n, \left[S_0 + S_\mu\right]^{-1}\right)$$

with $S_\mu = \sum_{i=1}^{n} (y_i - \mu)\,(y_i - \mu)^T$.

```
# Function to perform Bayesian estimation of mean and covariance using Gibbs
sampler

gibbssampler <- function(data, n_iter, prior_mean_mean, prior_mean_cov,
```

```
prior_cov_df, prior_cov_cov){

# Number of observations

n <- nrow(data)

# Initialize vectors to store samples

samples_mean  <- matrix(0, n_iter, 2)
samples_cov   <- array(0, dim = c(2, 2, n_iter))

# Initial values

current_mean <- colMeans(data)
current_cov  <- cov(data)

for (iter in 1:n_iter) {
  # Update mean
  precision <- solve(current_cov)
  mean_precision <- solve(n*precision + solve(prior_mean_cov))
  mean_mean <- mean_precision %*% (precision  %*% colSums(data)  +
solve(prior_mean_cov) %*% prior_mean_mean)
  current_mean <- mvrnorm(1, mean_mean, mean_precision)

  # Update covariance
  shape       <- prior_cov_df + n
  scale       <- solve(crossprod(data - matrix(1, n, 1) %*% t(current_mean))
+ prior_cov_cov)
  current_cov <- riwish(shape, scale)

  # Store samples
  samples_mean[iter, ]   <- current_mean
  samples_cov [, , iter] <- current_cov
}

# Return the samples
return(list(samples_mean = samples_mean, samples_cov = samples_cov))}
```

Now, we can run this chain for 10000 iterations and obtain a posterior estimates of the mean and covariance matrix.

```
library(MCMCpack)  # Package for IW
```

```
Loading required package: coda
```

```
##
## Markov Chain Monte Carlo Package (MCMCpack)
```

```
## Copyright (C) 2003-2025 Andrew D. Martin, Kevin M. Quinn, and Jong Hee
Park
```

```
##
## Support provided by the U.S. National Science Foundation

## (Grants SES-0350646 and SES-0350613)
##

library(MASS)

# Number of iterations

n_iter <- 10000

# Perform Bayesian estimation using Gibbs sampler

results <- gibbssampler(y, n_iter, prior_mean_mean, prior_mean_cov,
prior_cov_df, prior_cov_cov)

# Print the mean and covariance samples

print("Mean Samples:")
```

```
[1] "Mean Samples:"
```

```
head(results$samples_mean,n=3)
```

```
          [,1]     [,2]
[1,] -1.043617 1.008706
[2,] -1.032208 1.036881
[3,] -1.032273 1.036867
```

```
print("Covariance Samples:")
```

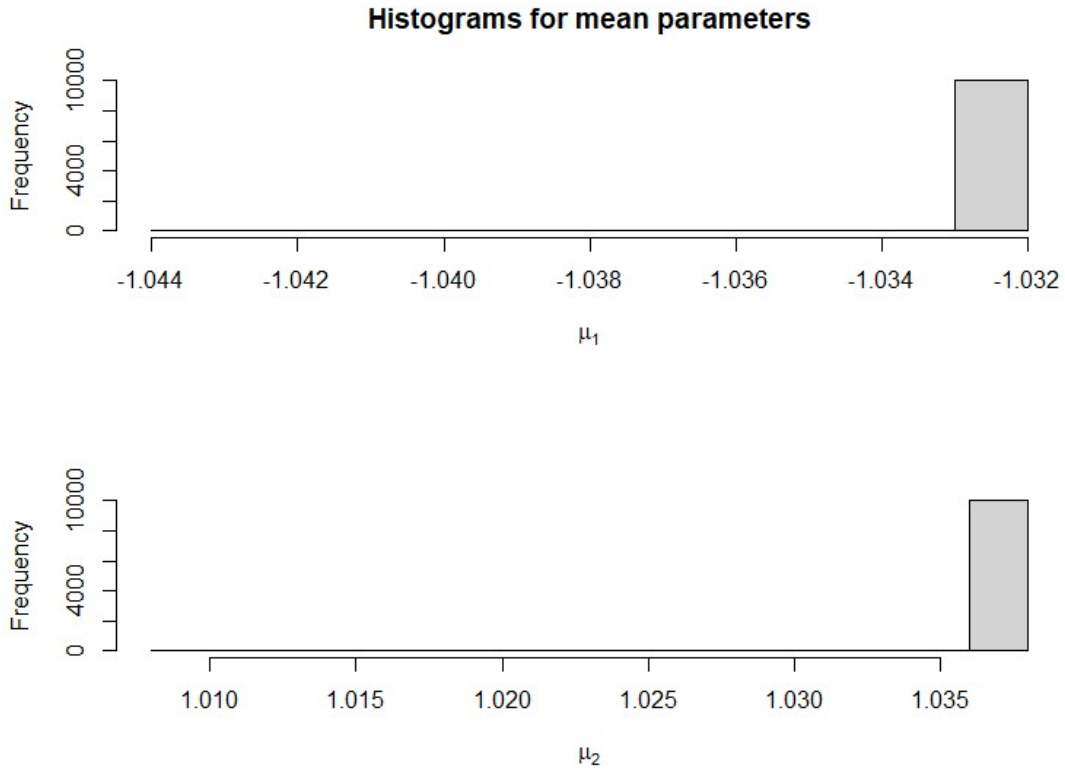
```
[1] "Covariance Samples:"
```

```
results$samples_cov[,,1]
```

```
              [,1]          [,2]
[1,]  1.124236e-06 -1.321848e-07
[2,] -1.321848e-07  3.878640e-07
```

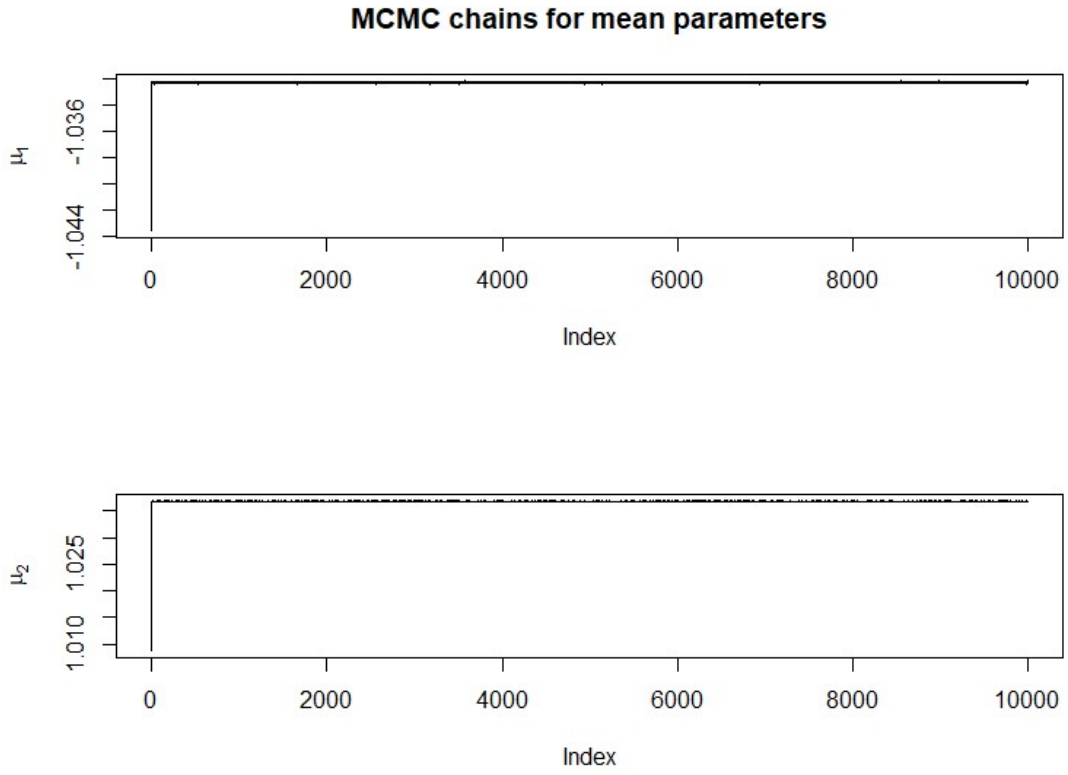
First, we can always depict a histogram of the sampling. For the mean parameters $\mu = (\mu_1, \mu_2)$, we have:

```
library(latex2exp)

# Plot histograms for the mean parameters

par(mfrow = c(2, 1))
hist(results$samples_mean[, 1], xlab = TeX(r"($\mu_1$)"),
     main = 'Histograms for mean parameters')
hist(results$samples_mean[, 2], xlab = TeX(r"($\mu_2$)"),main = '')
```



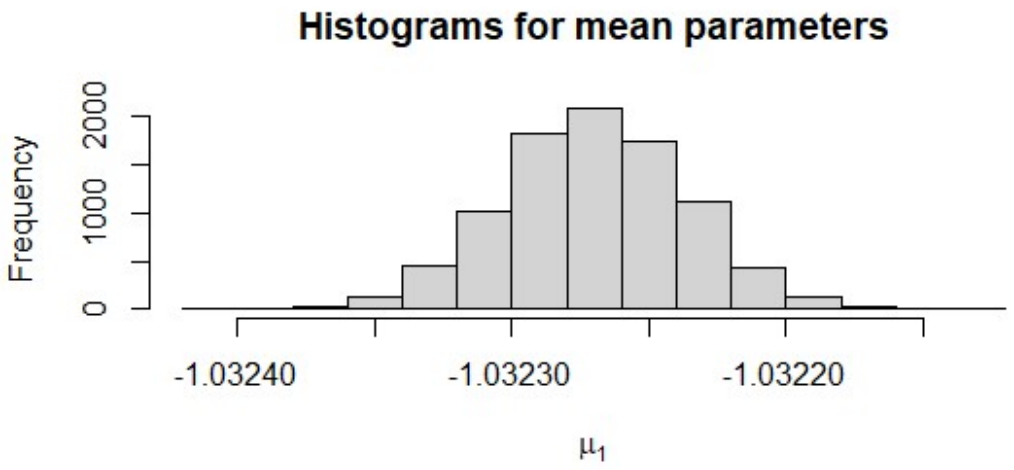
And the traces are:

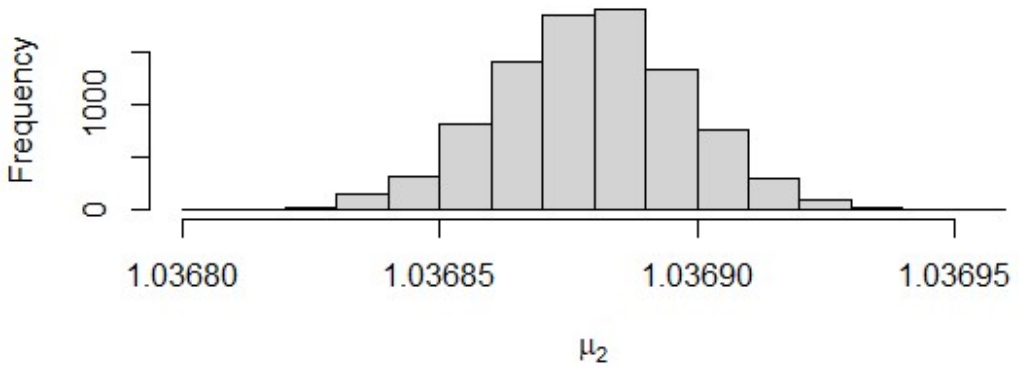
```
# Plot MCMC chains for the mean parameters

par(mfrow = c(2, 1))
plot(results$samples_mean[, 1], type = 'l', ylab = TeX(r"($\mu_1$)"),
     main = 'MCMC chains for mean parameters')
plot(results$samples_mean[, 2], type = 'l', ylab = TeX(r"($\mu_2$)"))
```



It clearly shows that the burn-in period was not excluded. Let us exclude the first 1000 iterations and depict the histograms of the means:

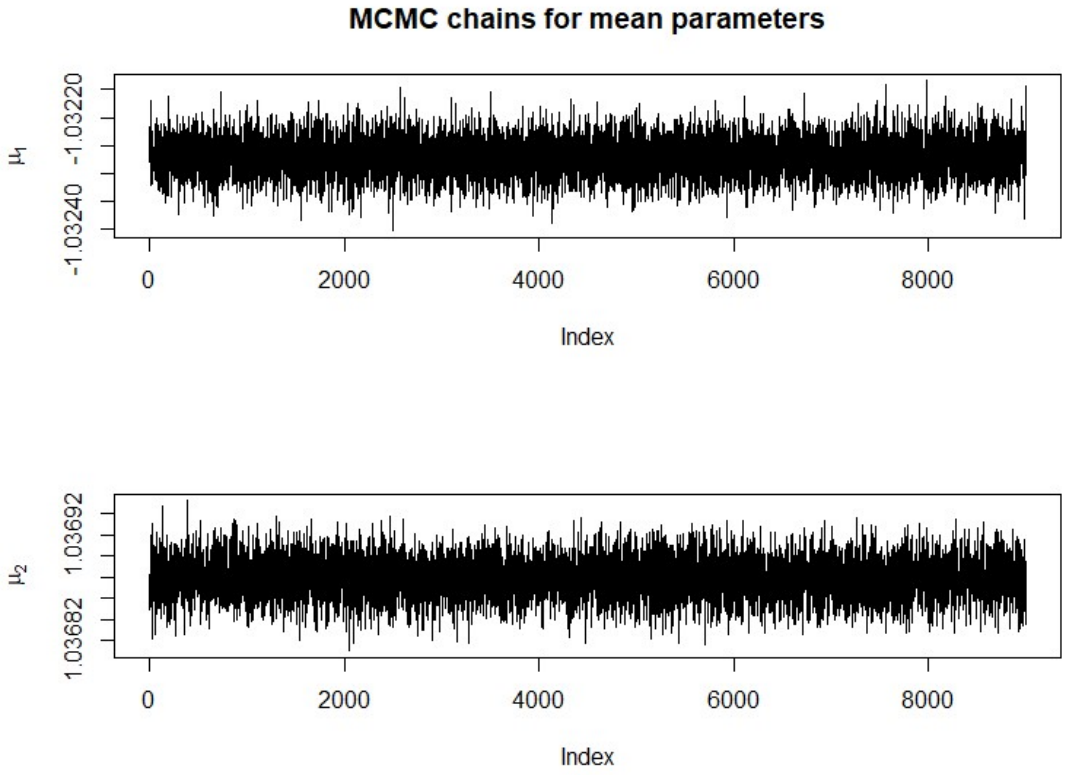
```
results$samples_mean <- results$samples_mean[1001:n_iter,]
results$samples_cov  <- results$samples_cov[,,1001:n_iter]

# Plot histograms for the mean parameters

par(mfrow = c(2, 1))
hist(results$samples_mean[, 1], xlab = TeX(r"($\mu_1$)"),
     main = 'Histograms for mean parameters')
hist(results$samples_mean[, 2], xlab = TeX(r"($\mu_2$)"),main = '')
```





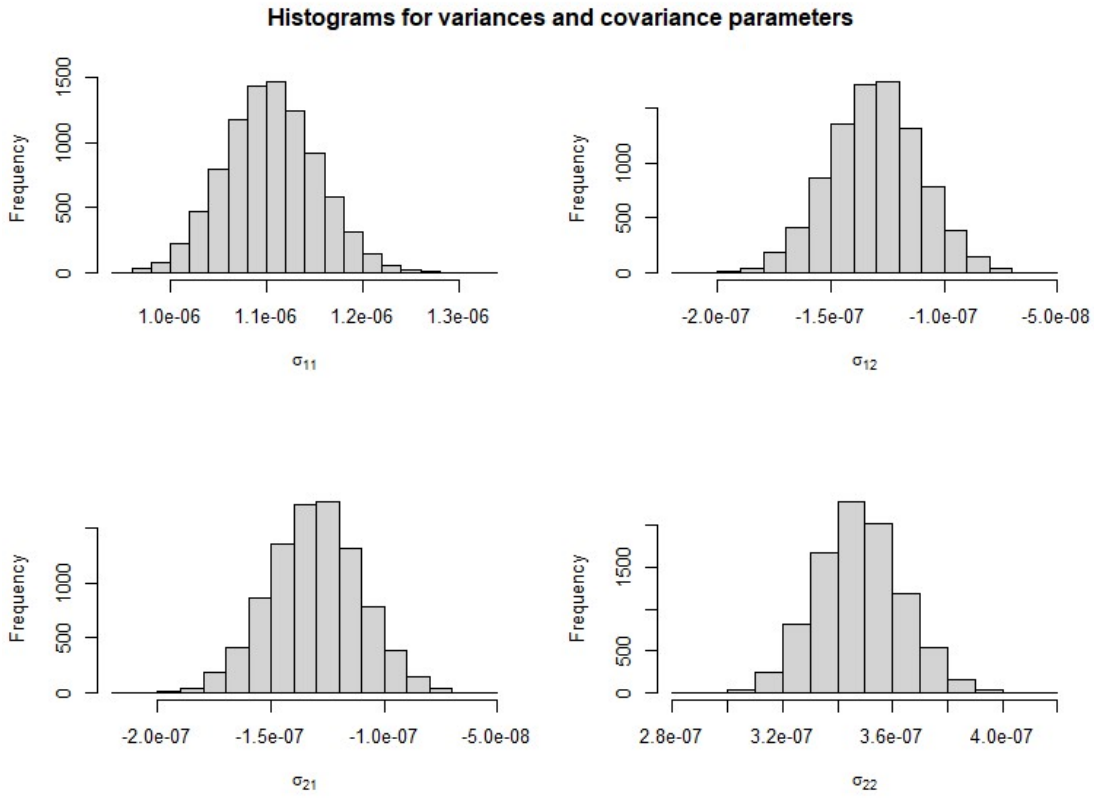
And the traces are:

```
# Plot MCMC chains for the mean parameters

par(mfrow = c(2, 1))
plot(results$samples_mean[, 1], type = 'l', ylab = TeX(r"($\mu_1$)"),
     main = 'MCMC chains for mean parameters')
plot(results$samples_mean[, 2], type = 'l', ylab = TeX(r"($\mu_2$)"))
```



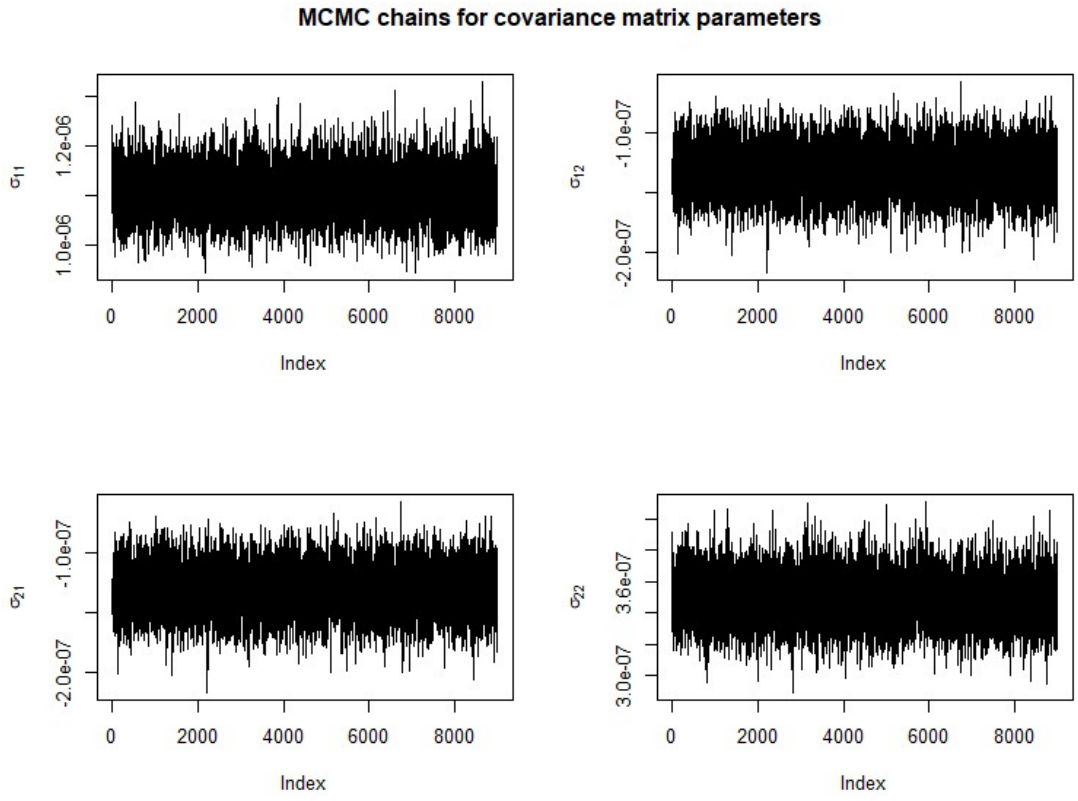
In the case of variances and covariance, we have:

```
# Plot histograms for variances and covariance

par(mfrow = c(2, 2))
for (i in 1:2) {
  for (j in 1:2) {
    aux <- paste("$ \\sigma_{",i,j,"}$")
    hist(results$samples_cov[i, j, ], xlab = TeX(aux),main = '')}
}
title(main='Histograms for variances and covariance parameters',outer=T,
line=-2)
```



And the traces are:

```
# Plot MCMC chains for the covariance matrix parameters

par(mfrow = c(2, 2))
for (i in 1:2) {
  for (j in 1:2) {
    aux <- paste("$ \\sigma_{",i,j,"}$")
    plot(results$samples_cov[i, j, ], type = 'l', ylab = TeX(aux))}
}
title(main='MCMC chains for covariance matrix parameters',outer=T, line=-2)
```



This example illustrates the black box of Stan (different MCMC algorithm). Next, we estimate a similar model using Stan.

4. Estimation using Stan

We can use Stan to estimate this model by sampling from the posterior (notice the slightly different parameterization of the priors):

```
model_stan <- "
data {
  int<lower=1> N;          // Number of observations
  int<lower=1> K;          // Number of dimensions
  matrix[N, K] y;          // Data matrix
}

parameters {
    vector[K] mu;                 // locations of the marginal normal
distributions
    //real mu2;                   // locations of the marginal normal
distributions
    real<lower=0> sigma1;         // scales of the marginal normal
distributions
    real<lower=0> sigma2;         // scales of the marginal normal
```

```
distributions
    real<lower=-1, upper=1> rho; // correlation coefficient
}

transformed parameters {
    // Covariance matrix
    matrix[K,K] cov;
    cov[1,1] = sigma1 ^ 2;
    cov[1,2] = sigma1 * sigma2 * rho;
    cov[2,1] = cov[1,2];
    cov[2,2] = sigma2 ^ 2;
}

model {
  // Likelihood
  // Bivariate normal distribution
  for (n in 1:N) {
        y[n] ~ multi_normal(mu, cov);
  }

  // Noninformative priors on all parameters
  mu     ~ normal(0, 100);
  sigma1 ~ normal(0, 100);
  sigma2 ~ normal(0, 100);
  rho    ~ normal(0, 100);
}"
```

Great, now we can sample data points from the posterior distribution of the parameters.

```
#install.packages("rstan", dependencies = TRUE)
library(rstan)
```

```
Loading required package: StanHeaders
```

```
rstan version 2.32.6 (Stan version 2.32.2)
```

```
For execution on a local, multicore CPU with excess RAM we recommend calling
options(mc.cores = parallel::detectCores()).
To avoid recompilation of unchanged Stan programs, we recommend calling
rstan_options(auto_write = TRUE)
For within-chain threading using `reduce_sum()` or `map_rect()` Stan
functions,
change `threads_per_chain` option:
rstan_options(threads_per_chain = 1)
```

```
Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file
```

```
Attaching package: 'rstan'
```

```
The following object is masked from 'package:coda':

    traceplot
```

```
set.seed(123)


# Compile the Stan model

model <- stan_model(model_code = model_stan)

# Your data

data <- list(N = 1000, K=2, y = y)

# Run the Stan model

stan_fit <- sampling(model, data = data, chains = 4,
                     iter = 3000, warmup = 1000)
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.000724 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take
7.24 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 4.12 seconds (Warm-up)
Chain 1:                8.865 seconds (Sampling)
Chain 1:                12.985 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
```

```
Chain 2: Gradient evaluation took 0.000496 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take
4.96 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 4.488 seconds (Warm-up)
Chain 2:                10.481 seconds (Sampling)
Chain 2:                14.969 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 0.000537 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take
5.37 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 3: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 3: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 3: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 3: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 3: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 3: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 3: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 3: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 3: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 3: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 3: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 4.058 seconds (Warm-up)
Chain 3:                9.194 seconds (Sampling)
Chain 3:                13.252 seconds (Total)
Chain 3:
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 0.001158 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take
11.58 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 4: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 4: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 4: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 4: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 4: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 4: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 4: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 4: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 4: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 4: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 4: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 4.398 seconds (Warm-up)
Chain 4:                9.104 seconds (Sampling)
Chain 4:                13.502 seconds (Total)
Chain 4:
```

```
post <- as.array(stan_fit)
```

5. MCMC chain convergence

A Markov chain generates draws from the target distribution only after it has converged to the equilibrium, which is only guaranteed asymptotically. In practice, convergence diagnostics must be applied to monitor whether the Markov chains have converged.

The bayesplot package provides various plotting functions for visualizing Markov chain Monte Carlo (MCMC) diagnostics after fitting a Bayesian model.

There are several ways to measure the convergence of MCMC chains. This section discusses the most important ones.

5.1 Trace Plots

Visual inspection of trace plots can provide a quick assessment of convergence. Trace plots show how the parameter values change over iterations. A well-converged chain should exhibit random, stationary behavior. In the case of our example, we have for means

```
library(posterior)
```

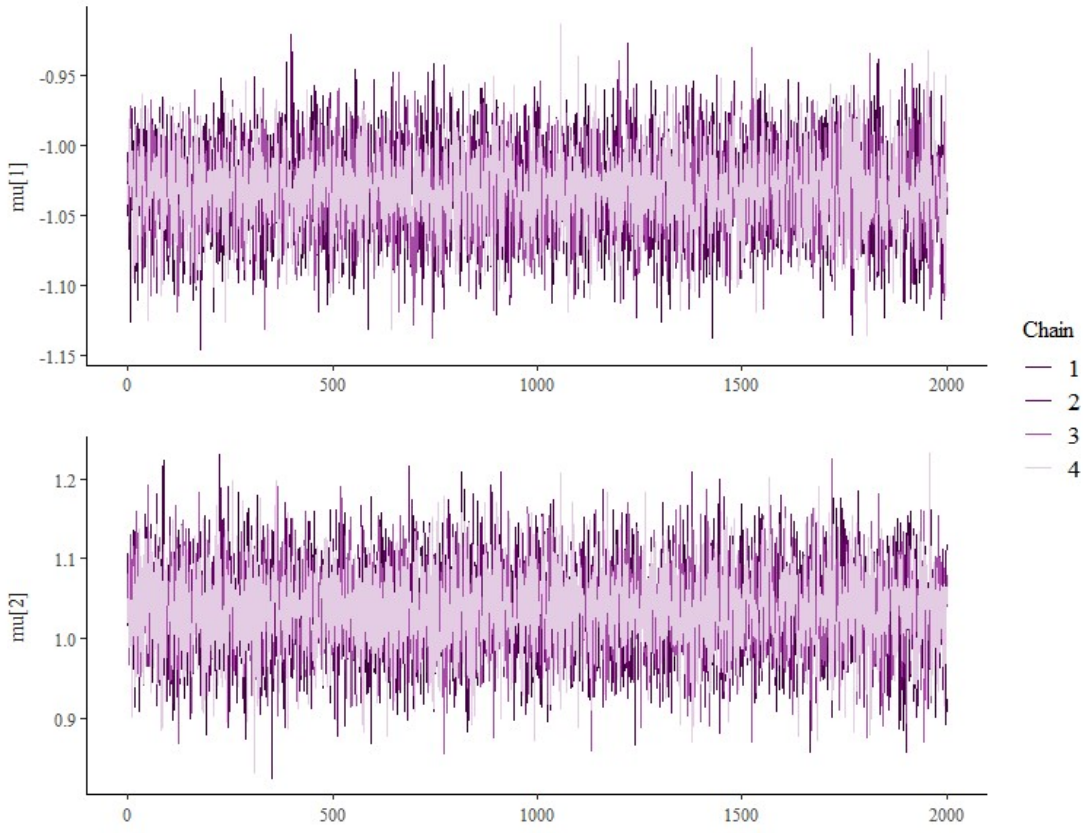
```
This is posterior version 1.5.0
```

```
Attaching package: 'posterior'

The following objects are masked from 'package:rstan':

    ess_bulk, ess_tail

The following objects are masked from 'package:stats':

    mad, sd, var

The following objects are masked from 'package:base':

    %in%, match
```

```
library(bayesplot)
```

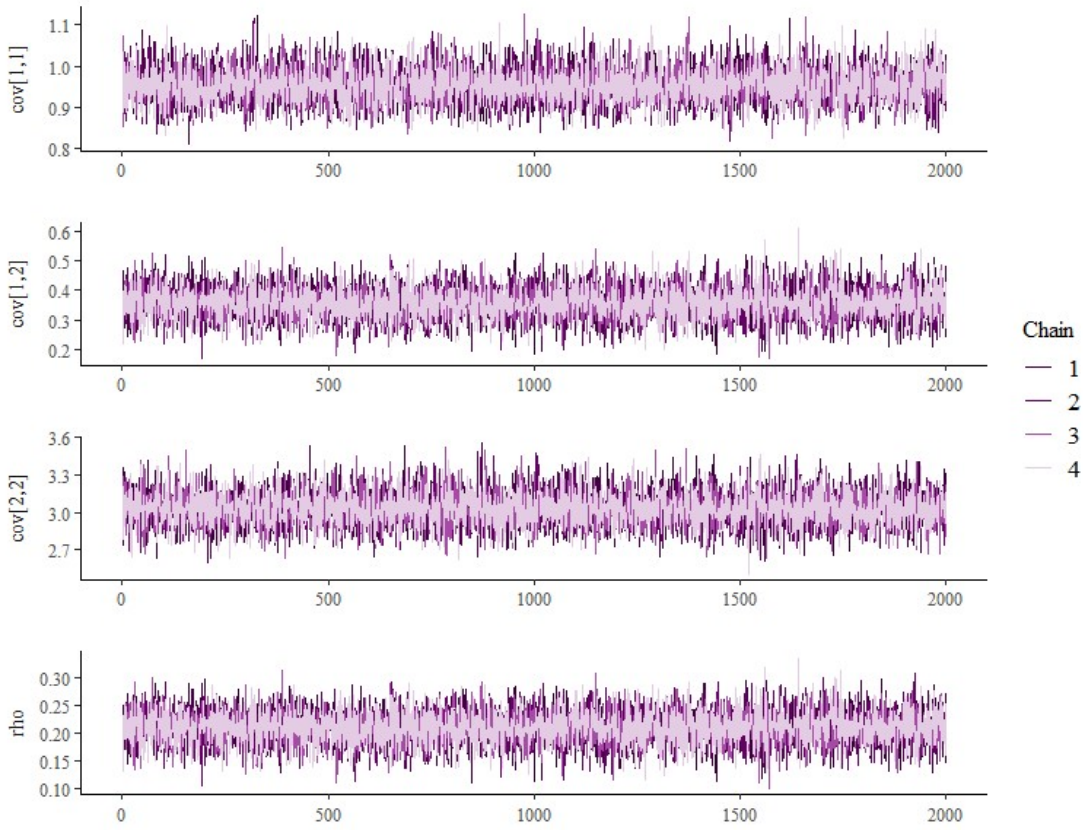
```
This is bayesplot version 1.11.1

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to bayesplot::theme_default()

   * Does _not_ affect other ggplot2 plots

   * See ?bayesplot_theme_set for details on theme setting


Attaching package: 'bayesplot'

The following object is masked from 'package:posterior':

    rhat
```

```
library(coda)
color_scheme_set("purple")

mcmc_trace(post, pars = c("mu[1]", "mu[2]"),
           facet_args = list(ncol = 1, strip.position = "left"))
```



and for the variances and covariances $\Sigma = \begin{pmatrix} \sigma_{11} = \sigma_1^2 & \sigma_{21} \\ \sigma_{21} & \sigma_{22} = \sigma_2^2 \end{pmatrix}$, and correlation the trace plots are:

```
#install.packages('rstan')
library(posterior)
library(bayesplot)
#install.packages("coda") # Install the coda package if not already installed
library(coda)
library(ggplot2)
color_scheme_set("purple")

mcmc_trace(post, pars = c("cov[1,1]", "cov[1,2]", "cov[2,2]", "rho"),
           facet_args = list(ncol = 1, strip.position = "left"))
```



5.2 Gelman-Rubin Statistic (R-hat)

The $\hat{R}$ or R-hat statistic compares the within-chain variance to the between-chain variance. The $\hat{R}$ statistic is calculated based on the potential scale reduction factor (PSRF), which is defined as the square root of the ratio of the total-chain variance to the within-chain variance, i.e.,

$$\hat{R} = \sqrt{\frac{\frac{n-1}{n}\widehat{W} + \frac{1}{n}\hat{B}}{\widehat{W}}} = \sqrt{1 - \frac{1}{n} + \frac{1}{n}\frac{\hat{B}}{\widehat{W}}}$$

where and are the estimated within-chain variance and estimated between-chain variance, respectively.

A value close to 1 indicates convergence. More specifically, if all chains are at equilibrium, these will be the same and $\hat{R}$ will be one; otherwise, the chains have not converged to a common distribution, and $\hat{R}$ statistic will be greater than one. The bayesplot package provides the functions mcmc_rhat and mcmc_rhat_hist for visualizing $\hat{R}$.

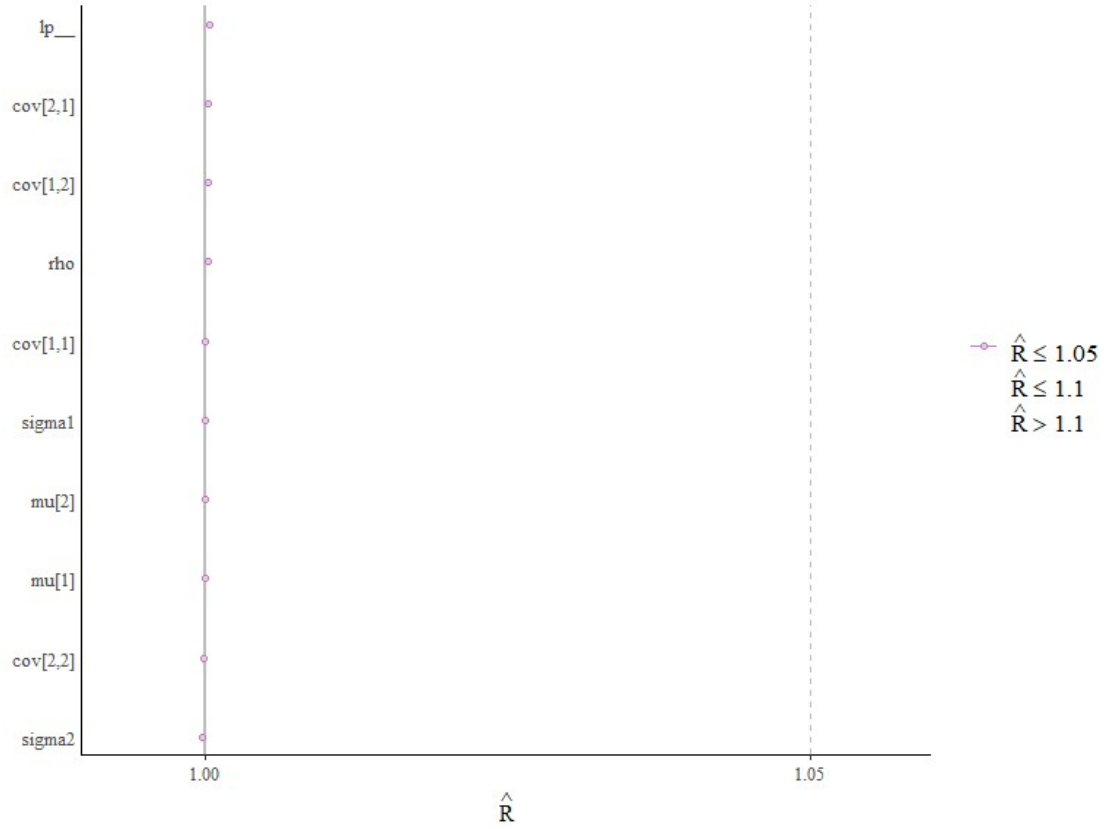
```
rhats <- rhat(stan_fit)
print(rhats)
```

```
    mu[1]     mu[2]     sigma1     sigma2        rho   cov[1,1]   cov[1,2]
cov[2,1]
```

```
0.9999171 0.9999325 0.9999943 0.9997735 1.0002018 1.0000034 1.0002052
1.0002052
 cov[2,2]      lp__
0.9997902 1.0002916
```

And graphically we observe excellent results:

```
mcmc_rhat(rhats) + yaxis_text(hjust = 1)
```



5.3 Effective Sample Size

The effective sample size is an estimate of the number of independent draws from the posterior distribution of the estimand of interest. Because the draws within a Markov chain are not independent if there is autocorrelation, the effective sample size, neff, is usually smaller than the total sample size, *N*. The larger the ratio of neff to N the better.

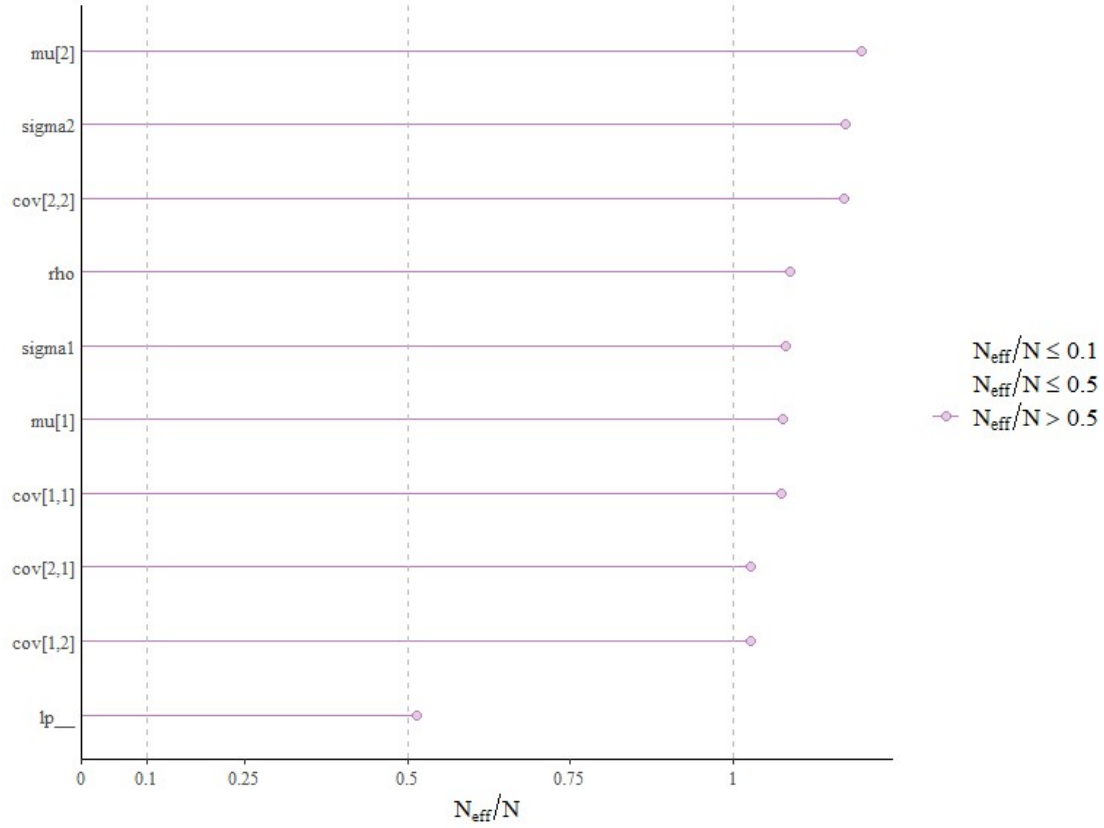
The bayesplot package provides a generic neff_ratio extractor function, currently with methods defined for models fit using the Stan (via rstan). The mcmc_neff can be used to plot the ratios.

```
ratios_cp <- neff_ratio(stan_fit)
print(ratios_cp)
```

```
    mu[1]     mu[2]    sigma1    sigma2       rho  cov[1,1]  cov[1,2]
cov[2,1]
```

```
1.0758334 1.1966669 1.0797884 1.1720290 1.0861659 1.0732584 1.0268586
1.0268586
 cov[2,2]      lp__
1.1694639 0.5135676
```

```
mcmc_neff(ratios_cp, size = 2) + yaxis_text(hjust = 1)
```



You should be worried about any ratio neff/N less than 0.1, which is not the case. The No-U-Turn sampler used by rstan generally produces much lower autocorrelations that for instance Gibbs sampler.

Note that these two important indicators are given in the summary

```
# Print and plot a summary of the Stan fit
```

```
print(stan_fit)
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=3000; warmup=1000; thin=1;
post-warmup draws per chain=2000, total post-warmup draws=8000.

             mean se_mean   sd     2.5%      25%      50%      75%    97.5%
mu[1]       -1.03    0.00 0.03    -1.09    -1.05    -1.03    -1.01    -0.97
mu[2]        1.04    0.00 0.05     0.93     1.00     1.04     1.07     1.15
sigma1       0.98    0.00 0.02     0.93     0.96     0.98     0.99     1.02
```

```
sigma2       1.74    0.00 0.04     1.66     1.71     1.74     1.77     1.82
rho          0.21    0.00 0.03     0.15     0.19     0.21     0.23     0.27
cov[1,1]     0.95    0.00 0.04     0.87     0.92     0.95     0.98     1.04
cov[1,2]     0.36    0.00 0.06     0.25     0.32     0.36     0.39     0.47
cov[2,1]     0.36    0.00 0.06     0.25     0.32     0.36     0.39     0.47
cov[2,2]     3.03    0.00 0.14     2.77     2.94     3.03     3.12     3.31
lp__      -1505.05    0.02 1.56 -1508.89 -1505.86 -1504.75 -1503.90 -1502.96
          n_eff Rhat
mu[1]      8607    1
mu[2]      9573    1
sigma1     8638    1
sigma2     9376    1
rho        8689    1
cov[1,1]   8586    1
cov[1,2]   8215    1
cov[2,1]   8215    1
cov[2,2]   9356    1
lp__       4109    1

Samples were drawn using NUTS(diag_e) at Wed Feb 26 15:53:24 2025.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

5.4 Autocorrelation

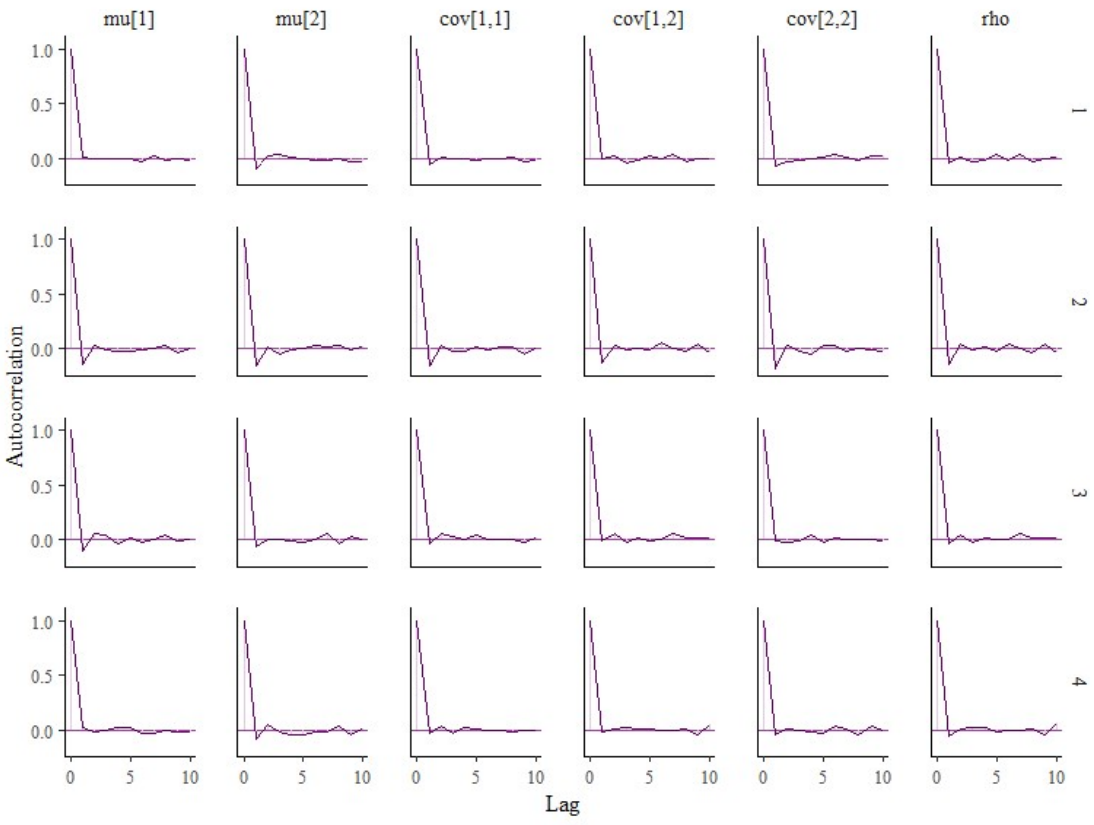
Autocorrelation is a measure of the correlation between consecutive samples in a Markov Chain Monte Carlo (MCMC) chain. The autocorrelation function (ACF) at lag k for a parameter θ in an MCMC chain is defined as:

$$ACF_k = \frac{cov(\theta_t, \theta_{t+k})}{\sqrt{var(\theta_t) var(\theta_{t+k})}},$$

where θ_x is the value of parameter θ at time t, θ_{t+k} is the value of parameter θ at time $t + k$, $cov(\theta_t, \theta_{t+k})$ is the covariance between θ_t and θ_{t+k}; and $var(\theta_t)$ and $var(\theta_{t+k})$ are the variances of θ_t and θ_{t+k}, respectively.

The autocorrelation function can take values between -1 and 1. Smaller autocorrelation values are preferred, because large autocorrelation in the chains may indicate slow mixing. We can visualize the autocorrelation using the mcmc_acf (line plot) or mcmc_acf_bar (bar plot) functions. For the selected parameters, these functions show the autocorrelation for each Markov chain separately up to a user-specified number of lags. Positive autocorrelation is bad (it means the chain tends to stay in the same area between iterations) and you want it to drop quickly to zero with increasing lag. Negative autocorrelation is possible and it is useful as it indicates fast convergence of sample mean towards true mean.

```
mcmc_acf(post, pars = c("mu[1]", "mu[2]", "cov[1,1]", "cov[1,2]", "cov[2,2]",
"rho"), lags = 10)
```



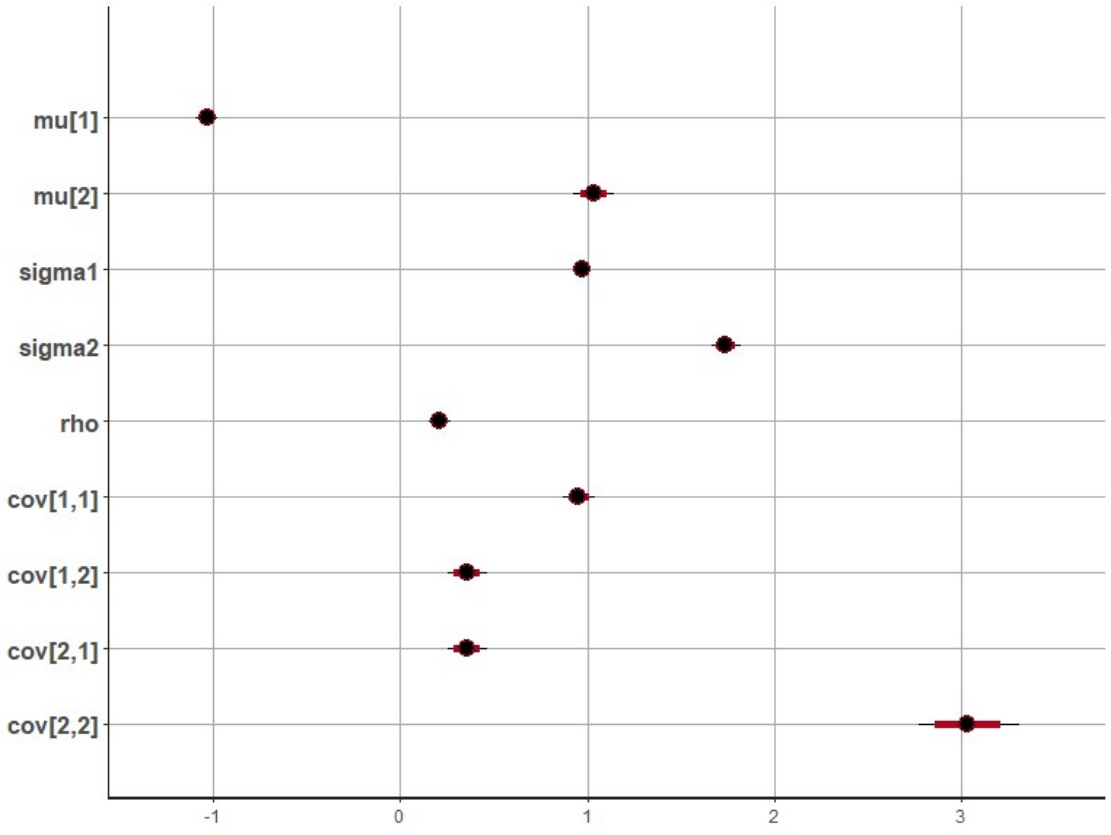
As mentioned above, neff/N decreases as autocorrelation becomes more extreme. The No-U-Turn sampler used by Stan generally produces much lower autocorrelations than for instance Gibbs sampler.

6. Conclusion

In summary, the next plot summarizes the estimates of the parameters in this model:

```
# Print and plot a summary of the Stan fit

plot(stan_fit)
```

```
ci_level: 0.8 (80% intervals)

outer_level: 0.95 (95% intervals)
```


mu[1]
mu[2]
sigma1
sigma2
rho
cov[1,1]
cov[1,2]
cov[2,1]
cov[2,2]
-1
0
1
2
3